Convolutional Neural Network Pipeline on a 1024x1024 Input with a 4x4 Kernel

Michael Ibeto
ECE Department, NCSU.

INTRODUCTION

Convolution is a key computational stage in modern image-processing and neural-network accelerators. This project implements a hardware 4×4 convolution engine that operates directly on a 1024×1024 image stored in external DRAM. Kernel weights and input tiles are fetched through burst-based DRAM reads, where addresses are issued once per burst and held across multiple cycles to hide memory latency and maintain sustained throughput. All convolution operations are performed using a serialized signed multiply-accumulate data path, allowing sixteen kernel products to be computed over multiple cycles using a single arithmetic unit. Partial sums are buffered locally to enable data reuse as the 4×4 window shifts across the image, and final results are saturated to 8-bit precision before being written back to DRAM using burst write commands. This design emphasizes low hardware cost and timing reliability while maintaining fully streamed memory access suited for integration into larger accelerator pipelines.

DESIGN DESCRIPTION

DATAPATH

The Datapath stores the entire 4×4 kernel and a 4×4 input window in locally buffered registers to avoid repeated DRAM accesses, enabling fast sliding-window computation. A single signed MAC unit iterates through the kernel and input window elements sequentially, accumulating partial sums over multiple cycles to produce one output pixel per window position. Intermediate sums are stored in dedicated registers so that multiple column positions can be computed in flight as the window shifts horizontally across the image. Once accumulation is complete, results are saturated to 8-bit signed range and written back to DRAM in burst format to maintain memory bandwidth efficiency. This architecture significantly reduces area and simplifies routing at the cost of requiring more cycles.

FINITE STATE MACHINE

The controller operates as a multi-phase finite state machine that sequences DRAM transactions, data buffering, MAC operations, and output writeback. After receiving a start signal, the FSM first loads the 4×4 kernel using two burst reads and stores the values into on-chip registers. It then iteratively loads input tiles from DRAM, buffering two rows at a time to form the sliding 4×4 convolution window. During computation states, the FSM selects kernel and input values in sequence and feeds them into a single MAC Data path, storing intermediate partial sums across multiple columns. Once all kernel rows are processed, the FSM finalizes results using saturation logic and issues a burst write to store output values back to DRAM. Row and column counters drive progression across the 1024×1024 image, allowing the FSM to reuse the same control flow for every window. The design prioritizes deterministic control and efficient burst-based memory access while minimizing redundant data fetches.

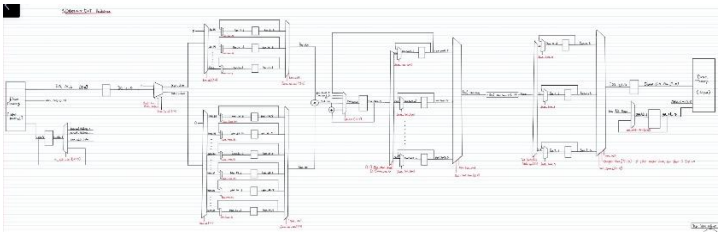
Results

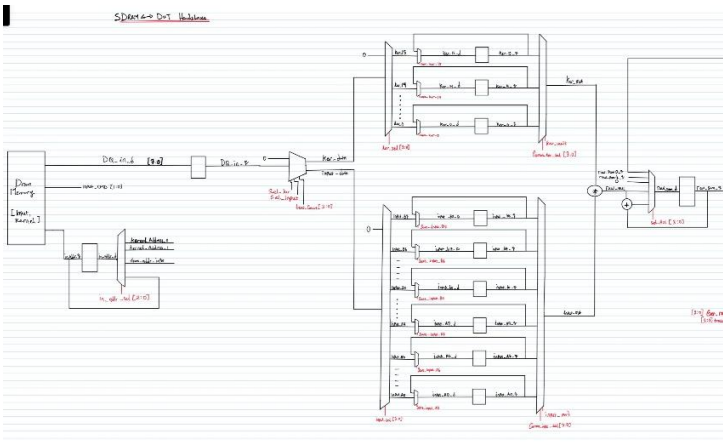
The implemented 4×4 convolution engine successfully performs convolution across the full 1024×1024 input image using streamed DRAM access and a single-cycle sequential MAC data path. The design operates at the target clock frequency and produces output values that match the provided test vectors for kernel-based convolution. Burst-based DRAM transactions allowed efficient transfer of input tiles and kernel weights while minimizing memory overhead. Final saturated 8-bit results are written back to memory in aligned bursts, completing the full convolution pipeline. Key performance characteristics including clock period, cycle count, resource utilization, and timing slack are summarized in Table 1.

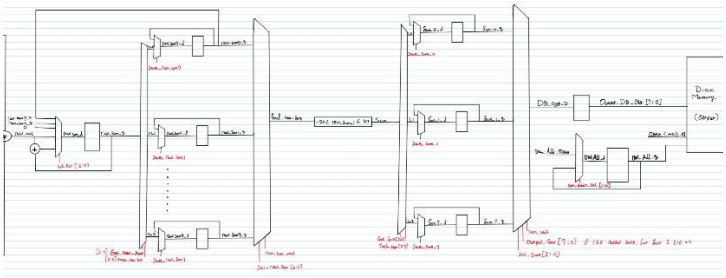
Conclusion

With additional refinement, performance could be improved by further pipelining the MAC accumulation path or reducing memory stalls during input window loading. A natural extension of this design would be parallelizing multiple MAC units to compute more output pixels per cycle, trading area for throughput. Similarly, integrating activation and pooling stages would allow the module to function as a complete CNN layer rather than a standalone convolution unit. Additional optimizations such as wider DRAM burst widths or input buffering via on-chip SRAM could further reduce latency and external memory dependence. Overall, the current implementation provides a functional, timing-closed hardware foundation suitable for scaling into a larger accelerator architecture.

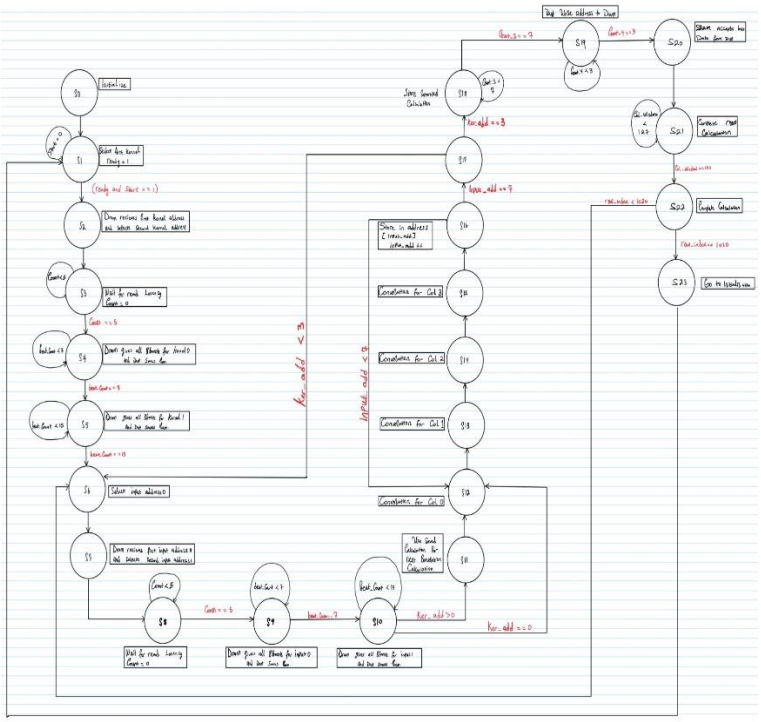
DATAPATH







FSM



Clock Period Achieved	6ns
Simulation Total Cycles (per individual test)	78,937,758 cycles
Total Cell Count	4019 cells
Total Cell Area	6659.044 μm^2
Setup Slack	0.0009
Hold Slack	0.0309

Design Achievements (Table 1)